

CAPÍTULO 02 — JAVA 21: CONTRATOS, COLECCIONES Y ERRORES CONTROLADOS

Aplicación guiada sobre las entidades de tu propio proyecto

GUÍA DEL ESTUDIANTE

Proyecto espejo ParkFlow 360	Ejemplo Cliente → Vehículo	Tecnología Java 21 puro
--	--------------------------------------	-----------------------------------

Tu misión

Observarás cómo el docente trabaja Cliente → Vehículo en ParkFlow 360. Tú NO debes copiar esas entidades. Debes aplicar el mismo razonamiento a las dos tablas 1:N que seleccionaste en tu proyecto: una entidad padre y una entidad dependiente.

1. Qué debes aprender

- Crear una interfaz como contrato.
- Usar List, Set y Map de manera consciente.
- Representar resultados opcionales con Optional.
- Crear excepciones propias.
- Usar enum para estados válidos.
- Usar record para datos simples e inmutables.
- Organizar tu código por responsabilidades.
- Explicar por qué tu código empieza a parecerse a una arquitectura hexagonal.

2. Antes de comenzar: identifica tus dos entidades

Elemento	Debes completar
Entidad padre	_____
PK de la entidad padre	_____
Entidad dependiente	_____
FK hacia la entidad padre	_____
Cardinalidad	1 : N

Atributo candidato a UNIQUE	_____
Atributo de estado, si existe	_____

Ejemplo del docente

ParkFlow: Cliente es la entidad padre. Vehículo es la entidad dependiente. Un cliente puede tener muchos vehículos y cada vehículo pertenece a un cliente.

3. Estructura mínima de paquetes que debes crear

```
src/main/java/com/tuempresa/tuproyecto/
├── <entidad-padre>/
│   ├── domain/
│   │   ├── TuEntidadPadre.java
│   │   ├── exception/
│   │   └── port/
│   ├── application/
│   │   └── TuEntidadPadreService.java
│   └── infrastructure/
│       └── memory/
│           └── TuEntidadPadreRepositoryEnMemoria.java
└── <entidad-dependiente>/
    ├── domain/
    │   ├── TuEntidadDependiente.java
    │   └── Estado....java
    ├── application/
    │   └── command/
    │       └── Registrar...Command.java
```

4. Paso 1 — crea el contrato del repositorio

Sigue el patrón del profesor, pero usa tu entidad real. Debes pensar qué operaciones necesita tu aplicación y expresarlas sin mencionar PostgreSQL, SQL ni JPA.

```
public interface TuEntidadPadreRepository {
    TuEntidadPadre guardar(TuEntidadPadre entidad);
    Optional<TuEntidadPadre> buscarPorId(Long id);
    List<TuEntidadPadre> listarTodos();
    boolean existePorCampoUnico(String valor);
}
```

Debes poder defender

¿Por qué esta interfaz no sabe dónde se guardan los datos? Porque define un contrato; la tecnología de persistencia se decidirá fuera del dominio/aplicación.

5. Paso 2 — crea una implementación en memoria con Map

```
public class TuEntidadPadreRepositoryEnMemoria
    implements TuEntidadPadreRepository {

    private final Map<Long, TuEntidadPadre> datos = new LinkedHashMap<>();

    @Override
    public TuEntidadPadre guardar(TuEntidadPadre entidad) {
        datos.put(entidad.getId(), entidad);
        return entidad;
    }

    @Override
    public Optional<TuEntidadPadre> buscarPorId(Long id) {
        return Optional.ofNullable(datos.get(id));
    }

    @Override
    public List<TuEntidadPadre> listarTodos() {
        return new ArrayList<>(datos.values());
    }

    @Override
    public boolean existePorCampoUnico(String valor) {
        return datos.values().stream()
            .anyMatch(e -> e.getCampoUnico().equalsIgnoreCase(valor));
    }
}
```

Debes adaptar `getCampoUnico()` al atributo UNIQUE real de tu tabla: código, documento, correo, placa, número, etc.

6. Cómo elegir List, Set o Map

Si necesitas...	Usa	Razón
Mantener una secuencia y permitir repetidos	List	Representa una colección ordenada.
No permitir duplicados lógicos	Set	Solo conserva elementos distintos según igualdad.
Buscar por una clave	Map	Asocia clave → valor y simplifica la búsqueda por id/código.

Escribe tu decisión: En mi repositorio uso _____ porque _____.

7. Paso 3 — elimina null de las búsquedas

```
Optional<TuEntidadPadre> buscarPorId(Long id);
```

Una búsqueda puede encontrar o no encontrar. Optional hace visible esa posibilidad en el tipo de retorno.

```
TuEntidadPadre entidad = repositorio.buscarPorId(id)
    .orElseThrow(() -> new TuEntidadNoEncontradaException(id));
```

8. Paso 4 — crea excepciones de negocio

```
public class TuEntidadNoEncontradaException extends RuntimeException {
    public TuEntidadNoEncontradaException(Long id) {
        super("No existe la entidad con id: " + id);
    }
}

public class CampoUnicoDuplicadoException extends RuntimeException {
    public CampoUnicoDuplicadoException(String valor) {
        super("Ya existe un registro con el valor: " + valor);
    }
}
```

No copies el nombre de la excepción

Cámbialo para expresar tu dominio. Ejemplos: `CodigoProductoDuplicadoException`, `CorreoPacienteDuplicadoException`, `NumeroContratoDuplicadoException`.

9. Paso 5 — crea tu servicio Java puro

```
public class TuEntidadPadreService {

    private final TuEntidadPadreRepository repository;

    public TuEntidadPadreService(TuEntidadPadreRepository repository) {
        this.repository = repository;
    }

    public TuEntidadPadre registrar(TuEntidadPadre entidad) {
        if (repository.existePorCampoUnico(entidad.getCampoUnico())) {
            throw new CampoUnicoDuplicadoException(entidad.getCampoUnico());
        }
        return repository.guardar(entidad);
    }

    public TuEntidadPadre obtener(Long id) {
        return repository.buscarPorId(id)
            .orElseThrow(() -> new TuEntidadNoEncontradaException(id));
    }
}
```

```

    public List<TuEntidadPadre> listar() {
        return repository.listarTodos();
    }
}

```

10. Paso 6 — modela estados con enum

Si una de tus tablas tiene un estado con conjunto cerrado, crea un enum. Usa únicamente estados que existan en tu modelo/reglas del proyecto.

```

public enum EstadoTuEntidad {
    ACTIVO,
    INACTIVO
}

```

Importante

No inventes ACTIVO/INACTIVO si tu proyecto tiene otros estados. Copia los estados reales definidos en tu modelo físico o requisitos.

11. Paso 7 — crea un record para registrar tu entidad dependiente

```

public record RegistrarEntidadDependienteCommand(
    Long entidadPadreId,
    String campo1,
    String campo2
) {}

```

Tu record debe contener únicamente los datos que necesita esa operación. No agregues campos solo porque existen en la tabla.

12. Paso 8 — prueba todo desde Main

```

public class Main {
    public static void main(String[] args) {
        var repository = new TuEntidadPadreRepositoryEnMemoria();
        var service = new TuEntidadPadreService(repository);

        service.registrar(/* primer objeto válido */);
        service.registrar(/* segundo objeto válido */);

        System.out.println(service.listar().size());
        System.out.println(service.obtener(/* id existente */));

        try {
            service.obtener(999L);
        } catch (RuntimeException ex) {
            System.out.println("ERROR CONTROLADO: " + ex.getMessage());
        }
    }
}

```

```
}  
}
```

13. Qué debe ocurrir en tu ejecución

1. El proyecto compila.
2. Puedes registrar al menos dos objetos de la entidad padre.
3. `listar()` devuelve esos objetos.
4. `obtener(id existente)` devuelve el objeto correcto.
5. `obtener(id inexistente)` produce tu excepción propia.
6. Registrar dos valores `UNIQUE` repetidos debe provocar la excepción correspondiente.

14. Error que está prohibido

```
public class TuEntidadPadreService {  
    private final TuEntidadPadreRepositoryEnMemoria repository;  
}
```

Esto acopla el servicio a una implementación concreta. Debes depender de la interfaz:

```
private final TuEntidadPadreRepository repository;
```

15. Relaciona este capítulo con tu diseño PostgreSQL

PostgreSQL	Java en este capítulo
PRIMARY KEY	id del objeto / clave del Map
UNIQUE	regla verificada antes de guardar en memoria
FOREIGN KEY 1:N	id de entidad padre dentro de la dependiente
CHECK de estado	enum cuando el conjunto de estados es cerrado
NOT NULL	constructor/validación obligatoria en Java
Tabla	concepto de dominio; no es todavía <code>@Entity</code>

No confundir

La clase Java de dominio NO es todavía una tabla JPA. En capítulos posteriores aparecerán las entidades de persistencia y el mapeo con PostgreSQL.

16. Evidencia que debes subir al repositorio

- Interfaz de repositorio de la entidad padre.
- Implementación en memoria.
- Servicio Java puro.
- Dos excepciones propias como mínimo.

- Un enum cuando corresponda.
- Un record para una operación.
- Main con pruebas positivas y negativas.
- README con explicación de tus decisiones.

17. README mínimo

Capítulo 02 - Java 21

Entidad padre

...

Entidad dependiente

...

Relación

1:N porque ...

Colección elegida

Usamos Map porque ...

Optional

La búsqueda ... puede no encontrar ...

Excepciones propias

- ...

- ...

Enum

... representa ...

Evidencia

Capturas / salida de consola / explicación breve.

18. Commit sugerido

```
git add .
```

```
git commit -m "feat(java): aplicar interfaces colecciones optional y excepciones al dominio"
```

19. Preguntas de defensa individual

7. ¿Cuál es tu entidad padre y cuál la dependiente?
8. ¿Dónde está materializada la relación 1:N en tus clases?
9. ¿Qué interfaz creaste y qué contrato representa?
10. ¿Por qué el servicio depende de la interfaz?
11. ¿Por qué elegiste Map/List/Set?
12. ¿Qué búsqueda devuelve Optional y por qué?

13. ¿Qué situación dispara cada excepción propia?
14. ¿Qué enum creaste y qué error evita?
15. ¿Para qué usaste record?
16. ¿Qué parte de tu código cambiará cuando reemplacemos memoria por PostgreSQL?

20. Definition of Done

- No copiaste Cliente/Vehículo como solución de tu proyecto.
- Tu solución usa las tablas reales que seleccionaste.
- El proyecto compila sin errores.
- La interfaz está separada de su implementación.
- Usas Optional en búsquedas que pueden fallar.
- Tienes excepciones propias y significativas.
- Tu enum corresponde al dominio real.
- Tu record tiene un propósito concreto.
- Puedes defender cada decisión sin depender de IA ni leer el código.

21. Qué viene después

Capítulo 03

Entraremos por primera vez a Spring Boot: Spring Initializr, estructura del proyecto, @SpringBootApplication, inversión de control, inyección de dependencias, Bean y primer endpoint REST. El objetivo será levantar una aplicación real antes de conectarla con PostgreSQL.